

Universidad Torcuato Di Tella

Tecnología Digital VI

Banco de Preguntas Teóricas para el Parcial

1er Semestre 2026

Cómo usar este documento: Este banco cubre todos los temas teóricos evaluables en el parcial. Las preguntas marcadas con **[CLÁSICA]** son las que se repiten históricamente. Las respuestas aparecen en cajas verdes a continuación de cada pregunta.

Índice

Preprocesamiento y Validación

Valores Faltantes

- P1.** Explique los tres mecanismos de datos faltantes (MCAR, MAR, MNAR) y dé un ejemplo de cada uno.

MCAR (*Missing Completely at Random*): la ausencia no depende de ningún valor. Ejemplo: un sensor falla aleatoriamente.

MAR (*Missing at Random*): la ausencia depende de otras variables observadas, pero no del valor faltante en sí. Ejemplo: hombres no responden preguntas de salud con más frecuencia que mujeres.

MNAR (*Missing Not at Random*): la ausencia depende del propio valor faltante. Ejemplo: personas con ingresos muy altos no declaran su sueldo. Es el caso más problemático.

- P2.** ¿Qué estrategias existen para imputar valores faltantes? ¿Cuándo usar cada una?

Media/mediana/moda: simple, funciona bien con MCAR. La media es sensible a outliers; la mediana es más robusta.

Nueva categoría (para categóricas): modelar explícitamente la ausencia.

Imputación por modelo (KNN, regresión): más precisa pero costosa computacionalmente.

Importante: toda imputación introduce algún sesgo.

Data Leakage

- P3. [CLÁSICA]** ¿Qué es el *data leakage*? Dé un ejemplo concreto y explique cómo prevenirlo.

El **data leakage** ocurre cuando información del conjunto de test (o del futuro) se filtra al entrenamiento. Consecuencia: métricas artificialmente optimistas que no se reproducen en producción.

Ejemplos:

- Ajustar un StandardScaler sobre todo el dataset (train+test) antes de dividir.
- Imputar valores faltantes con la media de todo el dataset.
- Incluir variables que “saben el futuro” (ej: fecha de pago en un modelo de riesgo de impago).

Prevención: ajustar *siempre* los transformadores solo con datos de entrenamiento y luego aplicarlos al test. Usar pipelines.

- P4.** Un equipo aplica un MinMaxScaler ajustado sobre todo el dataset y luego divide en train/test. ¿Hay data leakage? Justifique.

Sí, hay data leakage. El scaler aprendió los mínimos y máximos del conjunto de test, de modo que el modelo ya “vió” información del test durante el preprocesamiento. El flujo correcto es: dividir primero, luego ajustar el scaler solo en train y transformar train y test con ese scaler.

Escalado de Features

- P5.** ¿Cuándo es obligatorio escalar features y cuándo no importa? Dé ejemplos de modelos en cada categoría.

Necesario: modelos basados en distancias (KNN, SVM) o gradiente descendente (regresión logística, redes neuronales). Sin escalado, features en escalas grandes dominan artificialmente.

No necesario: árboles de decisión y sus ensambles (Random Forest, Gradient Boosting), ya que sus splits comparan valores del mismo feature, independientemente de la escala.

- P6.** Compare StandardScaler, MinMaxScaler y RobustScaler. ¿Cuándo preferiría cada uno?

StandardScaler: centra en 0 con $\sigma = 1$. Preferir cuando los datos son aproximadamente normales y no hay outliers extremos.

MinMaxScaler: escala al rango $[0, 1]$. Útil cuando se necesita un rango acotado, pero muy sensible a outliers.

RobustScaler: usa mediana e IQR. Preferir cuando hay outliers significativos, ya que no se deja afectar por ellos.

Desbalance de Clases

- P7. [CLÁSICA]** Un dataset de detección de fraude tiene 98 % de transacciones normales y 2 % fraudulentas. Un modelo que predice siempre “normal” obtiene 98 % de accuracy. ¿Qué problema hay y qué haría?

El accuracy es **engañoso** con clases desbalanceadas: un modelo trivial (siempre predice la clase mayoritaria) ya alcanza 98 % sin aprender nada.

Soluciones:

- Usar métricas adecuadas: F1-Score, Precision, Recall, AUC-PR.
- Oversampling de la clase minoritaria (SMOTE).
- Undersampling de la clase mayoritaria.
- Ajustar `class_weight` en el modelo.

- P8.** ¿Qué es SMOTE y cómo genera muestras sintéticas?

SMOTE (Synthetic Minority Oversampling Technique) genera muestras sintéticas de la clase minoritaria interpolando entre muestras existentes y sus vecinos más cercanos en el espacio de features. No duplica muestras sino que crea nuevas sintéticamente plausibles.

Validación Cruzada

- P9. [CLÁSICA]** Explique K-Fold Cross-Validation. ¿Por qué es preferible al simple holdout?

En **K-Fold CV** se divide el dataset en k particiones iguales. En cada iteración se usa un fold como validación y los $k - 1$ restantes como entrenamiento. Se promedian las k métricas resultantes.

Ventajas sobre holdout simple:

- Cada muestra es usada exactamente una vez para validar.
- La estimación del error es más robusta y con menor varianza.
- No depende de una única división aleatoria.

Valores típicos: $k = 5$ o $k = 10$.

P10. ¿Qué es Leave-One-Out CV (LOOCV)? ¿Cuál es su ventaja y su desventaja principal?

LOOCV es K-Fold con $k = n$ (una muestra por fold).

Ventaja: usa casi todos los datos para entrenar en cada iteración.

Desventaja: es computacionalmente muy costoso (n modelos a entrenar) y la varianza de la estimación es alta.

Métricas de Evaluación

Regresión

P11. Compare MSE, RMSE y MAE. ¿Cuándo elegiría cada uno?

MSE: penaliza cuadráticamente los errores grandes. Usar cuando equivocarse por mucho es muy costoso. Es diferenciable, estándar en entrenamiento.

RMSE: raíz del MSE; mismas propiedades pero en la unidad original. Comparar con MAE para detectar si hay puntos con error muy grande.

MAE: promedio de errores absolutos. Robusto a outliers (prioriza la mediana). Usar cuando se quiere una métrica directamente interpretable.

P12. ¿Puede el R^2 ser negativo? Explique qué significaría eso.

Sí, el R^2 puede ser negativo. Significa que el modelo es **peor** que simplemente predecir la media de los valores. $R^2 = 0$ equivale a predecir la media; $R^2 = 1$ es predicción perfecta.

P13. ¿Cuándo usar RMSLE en lugar de RMSE?

RMSLE es adecuado cuando importa el **error relativo (porcentual)** y no el absoluto. También cuando los targets abarcan varios órdenes de magnitud. Penaliza más las subestimaciones que las sobreestimaciones. Muy usado en forecasting de ventas o precios.

Clasificación

P14. [CLÁSICA] Defina Precision y Recall. ¿Cuándo es más grave tener Recall bajo? ¿Cuándo es más grave tener Precision baja?

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$

Recall bajo es más grave cuando los falsos negativos son catastróficos: detectar cáncer, fraude, fallas en reactores nucleares. “No debo perder casos positivos reales.”

Precision baja es más grave cuando los falsos positivos tienen alto costo: filtrar spam (marcar emails legítimos como spam), diagnósticos médicos invasivos. “No quiero molestar/perjudicar a quien no lo merece.”

P15. ¿Qué es el F1-Score y cuándo es más útil que el Accuracy?

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Es la media armónica entre Precision y Recall. Es útil cuando las clases están **desbalanceadas** y ambos tipos de error importan. El Accuracy puede ser engañoso porque un clasificador trivial puede alcanzarlo sin aprender nada.

P16. Explique la curva ROC y el AUC-ROC. ¿Qué significa un AUC de 0.5?

La **curva ROC** grafica TPR (Recall) vs. FPR para distintos umbrales de clasificación. El **AUC-ROC** es el área bajo esa curva.

- AUC = 1: clasificador perfecto.
- AUC = 0.5: equivalente a clasificar al azar (línea diagonal).
- AUC < 0.5: peor que el azar (modelo invertido).

No usar ROC cuando las clases están muy desbalanceadas; preferir la curva Precision-Recall.

P17. ¿Cuándo preferiría la curva Precision-Recall sobre la curva ROC?

Cuando el **dataset está muy desbalanceado** y la clase positiva es la de interés. La curva ROC puede resultar artificialmente optimista porque el AUC se ve inflado por la gran cantidad de verdaderos negativos (TN). La curva PR se focaliza solo en la clase positiva y es más informativa en estos casos.

Modelos Lineales

Regresión Lineal y Logística

P18. ¿Por qué la Regresión Logística es un modelo lineal si usa la función sigmoide?

La Regresión Logística es lineal porque la **frontera de decisión** es un hiperplano (combinación lineal de features = 0) y la relación entre features y log-odds es lineal:

$$\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$$

La sigmoide solo transforma la salida lineal a probabilidades (0, 1), pero no cambia la naturaleza lineal del modelo.

P19. ¿Por qué la Regresión Lineal tiene solución cerrada pero la Logística no?

La Regresión Lineal minimiza una función cuadrática (MSE) que admite solución exacta mediante las ecuaciones normales: $\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$.

La Logística minimiza la log-loss (cross-entropía), que no tiene solución cerrada debido a la no linealidad de la sigmoide. Se requieren métodos iterativos como Gradient Descent.

Regularización

P20. [CLÁSICA] Compare L1 (Lasso) y L2 (Ridge). ¿En qué se diferencian y cuándo usar cada una?

Propiedad	L1 (Lasso)	L2 (Ridge)
Penalización	$\lambda \sum w_j $	$\lambda \sum w_j^2$
Coefficientes	Exactamente 0 (esparsidad)	Pequeños, nunca 0
Selección de features	Sí	No
Multicolinealidad	Selecciona una	Distribuye peso
Solución	No analítica	Analítica

Usar L1 cuando se sospecha que muchas features son irrelevantes (selección implícita).

Usar L2 cuando todas las features son potencialmente útiles y hay multicolinealidad.

P21. ¿Qué efecto tiene aumentar λ en la regularización? ¿Y disminuirlo?

Aumentar λ : mayor penalización sobre los coeficientes $\Rightarrow$ modelo más simple $\Rightarrow$ mayor sesgo, menor varianza $\Rightarrow$ menos overfitting.

Disminuir λ : menor penalización $\Rightarrow$ modelo más complejo $\Rightarrow$ menor sesgo, mayor varianza $\Rightarrow$ más riesgo de overfitting.

Con $\lambda = 0$ no hay regularización.

Árboles de Decisión

P22. Explique intuitivamente por qué los árboles de decisión son modelos de **alta varianza**.

Los árboles de decisión son de alta varianza porque son muy sensibles a los datos de entrenamiento: un cambio pequeño en el dataset puede producir un árbol completamente diferente. Esto ocurre porque el algoritmo CART es **goloso (greedy)** y determinista: en cada nodo elige el mejor split para ese subconjunto, por lo que un punto extra o diferente puede cambiar todos los splits subsiguientes en cascada.

P23. Defina Gini y Entropía. ¿Qué valor toman cuando el nodo es puro? ¿Y cuando es máximamente impuro?

$$\text{Gini}_m = 1 - \sum_{k=1}^K p_{mk}^2, \quad H_m = - \sum_{k=1}^K p_{mk} \log_2(p_{mk})$$

Nodo puro (todos de la misma clase): Gini = 0, Entropía = 0.

Máxima impureza (clases equidistribuidas): Gini alcanza $1 - 1/K$ y la Entropía alcanza $\log_2(K)$.

P24. [CLÁSICA] Mencione **tres** hiperparámetros que regularizan un árbol de decisión (reducen overfitting) y explique el efecto de cada uno.

1. **max_depth**: limita la profundidad máxima del árbol. Menos profundidad $\Rightarrow$ menos particiones $\Rightarrow$ menos overfitting.
 2. **min_samples_split**: mínimo de muestras requeridas para dividir un nodo. Un valor alto impide crear nodos con pocos datos.
 3. **min_samples_leaf**: mínimo de muestras en cada hoja. Evita hojas con una sola muestra que memorizan el ruido.
- (También válido: **max_features**, **max_leaf_nodes**, **ccp_alpha** para poda.)

P25. Un árbol fue entrenado con temperaturas entre 10°C y 40°C. En producción recibe una observación de 55°C. ¿Qué predice? ¿Es un problema?

El árbol predecirá el valor de la **hoja más extrema** para temperaturas $> 40^\circ\text{C}$ (no puede extrapolar, solo puede asignar la última región). **Sí es un problema**: los árboles no extrapolan fuera del rango de entrenamiento. Si los nuevos datos presentan valores fuera del rango visto en train, el modelo tendrá un rendimiento degradado. Esto contrasta con los modelos lineales que sí extrapolan (aunque también con riesgo).

P26. ¿Qué es la importancia de features en un árbol? ¿Cuál es su sesgo principal?

La **importancia de features** suma la ganancia en impureza ponderada que aporta cada feature en todos los splits del árbol, normalizada para que sume 1.

Sesgo: los árboles tienden a sobreestimar la importancia de variables con alta cardinalidad o muchas categorías, porque tienen más oportunidades de hacer buenos splits por azar. Corrección: usar importancia por permutación.

Ensamblados I: Bagging y Random Forest

P27. Explique el concepto de **Bootstrap**. ¿Qué porcentaje de muestras queda fuera (OOB) en promedio?

El **Bootstrap** consiste en extraer n muestras **con reemplazo** de un dataset de n elementos. En promedio, $\approx 63,2\%$ de las muestras originales aparecen al menos una vez y el $\approx 36,8\%$ ($= 1/e$) queda fuera (*Out-of-Bag*, OOB), ya que:

$$\lim_{n \rightarrow \infty} \left(1 - \frac{1}{n}\right)^n = \frac{1}{e} \approx 0,368$$

P28. ¿Cómo se usa el error OOB en Random Forest? ¿Qué ventaja tiene frente a la validación cruzada?

Para cada muestra, el **error OOB** se estima usando solo los árboles que **no** la incluyeron en su bootstrap. Así se obtiene una estimación del error de generalización sin necesidad de reservar un conjunto de validación separado.

Ventaja: no desperdicia datos (todos se usan para entrenar) y es más eficiente computacionalmente que el K-Fold CV.

P29. [CLÁSICA] ¿Cuál es la diferencia fundamental entre Bagging y Random Forest?

Bagging entrena múltiples modelos en paralelo sobre muestras bootstrap independientes. Reduce la varianza promediando predictores.

Random Forest extiende Bagging con una capa adicional de aleatoriedad: en cada split solo se evalúan m features seleccionados al azar (típicamente $m = \sqrt{p}$ para clasificación, $m = p/3$ para regresión). Esto **decorrelaciona más** los árboles, reduciendo aún más la varianza del ensemble.

P30. ¿Agregar más árboles a un Random Forest puede causar overfitting? ¿Y en Gradient Boosting?

Random Forest: No. Agregar más árboles estabiliza o mejora el rendimiento sin causar overfitting. El error OOB converge a un valor estable.

Gradient Boosting: Sí. Como los árboles se entrenan secuencialmente para corregir los errores del anterior, agregar demasiados puede hacer que el modelo se ajuste al ruido del entrenamiento (overfitting). Por eso se usa *early stopping*.

P31. ¿Por qué la diversidad (decorrelación) entre modelos es importante en un ensemble?

Para B modelos con varianza σ^2 y correlación ρ :

$$\text{Var}(\bar{f}) = \rho \sigma^2 + \frac{1 - \rho}{B} \sigma^2$$

Si $\rho \approx 0$ (modelos decorrelacionados), la varianza del ensemble se reduce a σ^2/B , que tiende a 0 al aumentar B . Si ρ es alto, el beneficio de agregar modelos es mínimo. La clave es **reducir la correlación** entre modelos.

Ensamblados II: Boosting

P32. [CLÁSICA] Explique la diferencia conceptual entre Bagging y Boosting en cuanto a cómo reducen el error.

Bagging: entrena modelos en **paralelo** sobre muestras bootstrap independientes. Reduce la **varianza** del modelo promediando predictores diversos. No reduce mucho el sesgo.

Boosting: entrena modelos **secuencialmente**, donde cada modelo nuevo se focaliza en los errores del ensemble anterior. Reduce principalmente el **sesgo**, construyendo un modelo fuerte a partir de learners débiles.

P33. Explique el algoritmo de AdaBoost. ¿Cómo se actualizan los pesos?

1. Inicializar pesos iguales: $w_i = 1/n$.
2. Entrenar clasificador h_m con pesos $\{w_i\}$.
3. Calcular error ponderado: $\varepsilon_m = \sum_{h_m(x_i) \neq y_i} w_i$.
4. Peso del clasificador: $\alpha_m = \frac{1}{2} \ln \frac{1 - \varepsilon_m}{\varepsilon_m}$.
5. Aumentar pesos de muestras mal clasificadas; reducir las correctas.

6. Renormalizar pesos.

Predicción final: $H(x) = \text{sign}(\sum_m \alpha_m h_m(x))$.

Si $\varepsilon_m < 0,5$ entonces $\alpha_m > 0$ (clasificador mejor que el azar).

P34. ¿Qué son los pseudo-residuos en Gradient Boosting? ¿Por qué se usan?

Los **pseudo-residuos** son el gradiente negativo de la función de pérdida respecto de la predicción actual:

$$r_{im} = - \frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \Big|_{F=F_{m-1}}$$

Cada árbol nuevo se entrena para **predecir estos residuos**, lo que equivale a dar un paso en la dirección de mayor descenso de la pérdida en el espacio funcional. Para MSE, el pseudo-residuo es simplemente $y_i - \hat{y}_i$.

P35. ¿Qué rol cumplen el *learning rate* (η) y el número de árboles (M) en Gradient Boosting? ¿Cómo interactúan?

Learning rate η : controla cuánto contribuye cada árbol. Un η bajo hace pasos pequeños y más conservadores.

Número de árboles M : cuántos pasos se dan.

Trade-off: η pequeño requiere M grande para alcanzar buen desempeño, pero produce mayor regularización implícita y mejor generalización. Deben ajustarse en conjunto.

P36. Compare XGBoost, LightGBM y CatBoost en al menos dos dimensiones.

	XGBoost	LightGBM	CatBoost
Crecimiento	Por nivel	Por hoja	Simétrico
Velocidad	Moderada	Alta	Alta
Catóricas	Requiere enc.	Nativo	Nativo
Regularización	Explícita	Impl.+expl.	Implícita

LightGBM es más rápido pero más propenso a overfitting por su crecimiento *leaf-wise*. CatBoost maneja catóricas nativamente sin encoding previo.

Reducción de la Dimensionalidad

P37. ¿Qué es la maldición de la dimensionalidad y por qué afecta a los modelos basados en distancias?

En espacios de alta dimensión, el volumen crece exponencialmente. Los puntos se vuelven **equidistantes** entre sí: la diferencia entre la distancia máxima y mínima tiende a 0. Esto hace que los algoritmos basados en distancias (KNN, SVM radial, clustering) pierdan efectividad porque la noción de “vecino cercano” deja de ser significativa.

P38. Explique PCA paso a paso. ¿Qué maximiza y cómo se obtienen las componentes?

PCA busca una proyección lineal que **maximice la varianza retenida**.

1. Estandarizar cada feature (media 0, $\sigma = 1$).
2. Calcular la matriz de covarianza: $\mathbf{M}_x = \frac{1}{n-1} \mathbf{X}^\top \mathbf{X}$.
3. Descomponer: $\mathbf{M}_x \mathbf{v}_j = \lambda_j \mathbf{v}_j$ (autovectores y autovalores).
4. Ordenar por autovalor descendente.

5. Proyectar: $\mathbf{Z} = \mathbf{X} \mathbf{P}_k$.

La primera CP apunta a la dirección de máxima varianza; las siguientes son perpendiculares y tienen la segunda mayor varianza, etc.

P39. ¿Qué significa la varianza explicada en PCA? ¿Cómo se elige k ?

La varianza explicada por la componente j es:

$$VE_j = \frac{\lambda_j}{\sum_{l=1}^p \lambda_l}$$

Para elegir k (número de componentes a retener) se usa el **Scree Plot**: se busca el k donde la varianza acumulada supera un umbral deseado (típicamente 85–95%), o donde la curva muestra un “codo”.

P40. **[CLÁSICA]** Compare PCA, t-SNE y UMAP en cuanto a: linealidad, velocidad, y si pueden proyectar nuevas muestras.

	PCA	t-SNE	UMAP
Relaciones	Lineal	No lineal	No lineal
Velocidad	Muy rápida	Lenta	Rápida
Estructura local	Media	Alta	Alta
Estructura global	Alta	Baja	Media
Proyecta nuevas muestras	Sí	No	Sí
Interpretabilidad	Alta	Baja	Baja

t-SNE solo sirve para visualización exploratoria; no puede proyectar nuevas muestras. UMAP es preferido cuando se necesita velocidad y proyección de nuevas muestras.

Overfitting, Underfitting y Trade-offs

P41. **[CLÁSICA — SE TOMA SIEMPRE]** ¿Qué hiperparámetros pueden causar **overfitting**? Liste al menos cinco (de distintos modelos) y explique por qué.

1. **max_depth (árboles)**: valores altos permiten al árbol memorizar el ruido. Sin límite, cada muestra puede tener su propia hoja.
2. **n_estimators (Gradient Boosting)**: demasiados árboles en boosting hacen que el modelo se ajuste al ruido de training (en RF no causa overfitting).
3. **learning_rate bajo con n_estimators alto (GB)**: si la tasa es muy baja pero hay muchos árboles, el modelo puede sobreajustarse.
4. **min_samples_split / min_samples_leaf bajos (árboles)**: valores pequeños permiten splits con muy pocos datos, creando ramas que memorizan outliers.
5. **λ de regularización bajo (L1/L2)**: poca penalización permite coeficientes muy grandes que se ajustan al ruido.
6. **max_features alto (RF)**: si se usan muchas features por split, los árboles quedan más correlacionados y hay menos diversidad.

7. **Grado del polinomio (regresión polinomial)**: grados muy altos crean modelos que pasan exactamente por los datos de training.
8. **k pequeño en KNN**: $k = 1$ hace que el modelo simplemente memorice cada punto de entrenamiento.

P42. Explique el Bias-Variance Trade-off con la fórmula y un ejemplo.

$$\mathbb{E}[(y - \hat{f}(x))^2] = \underbrace{\left(\mathbb{E}[\hat{f}(x)] - f(x)\right)^2}_{\text{Sesgo}^2} + \underbrace{\mathbb{E}\left[\left(\hat{f}(x) - \mathbb{E}[\hat{f}(x)]\right)^2\right]}_{\text{Varianza}} + \underbrace{\sigma_\varepsilon^2}_{\text{Ruido irred.}}$$

Ejemplo: Un árbol muy profundo tiene bajo sesgo (se ajusta bien al train) pero alta varianza (cambia mucho ante nuevos datos $\Rightarrow$ overfitting). Un modelo lineal simple tiene alto sesgo (no captura patrones complejos $\Rightarrow$ underfitting) pero baja varianza.

P43. ¿Cómo detectar si un modelo tiene overfitting o underfitting a partir de las curvas de aprendizaje?

Overfitting: error de entrenamiento bajo pero error de validación alto (brecha grande entre ambas curvas). El modelo memoriza el train pero no generaliza.

Underfitting: error de entrenamiento y validación ambos altos (curvas convergentes en valores altos). El modelo no captura la señal de los datos.

Modelo adecuado: error de validación tiene un mínimo; elegir la complejidad en ese punto.

Selección de Features

P44. Compare los tres tipos de selección de features: Filter, Wrapper y Embedded.

Tipo	Método	Característica
Filter	Correlación, χ^2 , info. mutua	Independiente del modelo; rápido
Wrapper	Forward/Backward selection	Evalúa subconjuntos con modelo; lento
Embedded	L1 (Lasso), importancia en árboles	Integrado al entrenamiento

P45. ¿Por qué Label Encoding puede ser problemático para modelos lineales?

El Label Encoding asigna un número entero a cada categoría (ej: Argentina=0, Brasil=1, Chile=2), lo que **impone un orden ordinal artificial** que el modelo lineal interpretará como relación numérica. El modelo entenderá que “Chile > Brasil > Argentina”, lo cual no tiene sentido si las categorías no tienen orden. Solución: usar One-Hot Encoding para variables nominales.

Excepción: en árboles de decisión esto no es un problema, ya que los splits comparan valores y no asumen relaciones lineales.

Resumen: Comparación de Modelos

P46. [CLÁSICA] ¿Cuándo elegiría un árbol simple sobre Random Forest o Gradient Boosting?

Se preferiría un árbol simple cuando:

- La **interpretabilidad** es crítica (regulación, medicina, finanzas).
- Los recursos computacionales son muy limitados.
- El dataset es pequeño y los ensambles podrían no agregar valor.
- Se necesita un modelo de referencia (*baseline*) rápido.

En todos los demás casos, RF o GB suelen dar mejor performance.

P47. Complete la siguiente tabla comparativa de modelos:

Modelo	Sesgo	Varianza	Interpretable	No lineal
Reg. Lineal/Logística	Alto	Bajo	Muy alta	No
Árbol (profundo)	Bajo	Alto	Media	Sí
Árbol (podado)	Medio	Medio	Alta	Sí
Bagging / RF	Bajo	Bajo	Baja	Sí
Gradient Boosting	Muy bajo	Bajo	Baja	Sí

Preguntas de Integración

P48. Un equipo implementa el siguiente pipeline: (I) cargar dataset; (II) imputar con la media de todo el dataset; (III) escalar con MinMaxScaler sobre todo el dataset; (IV) dividir en train/test; (V) entrenar; (VI) evaluar. Identifique todos los errores.

Dos errores de data leakage:

1. La imputación en el paso II usa la media del test: introduce info del futuro.
2. El escalado en el paso III usa mínimos/máximos del test.

Pipeline correcto: (I) cargar; (II) dividir en train/test; (III) ajustar imputador solo en train, aplicar a ambos; (IV) ajustar scaler solo en train, aplicar a ambos; (V) entrenar; (VI) evaluar.

P49. Un modelo de detección de enfermedades tiene Recall = 0.99 pero Precision = 0.10. ¿Cómo lo interpretaría? ¿Es buen modelo?

Recall = 0.99: detecta el 99 % de los casos positivos reales (casi no pierde enfermos).
Precision = 0.10: de los que diagnostica como enfermos, solo el 10 % realmente lo está (90 % son falsos positivos).

El modelo es útil si el costo de no detectar una enfermedad es muy alto. Sin embargo, la Precision baja genera muchos diagnósticos falsos que pueden causar tratamientos innecesarios. Se debería ajustar el umbral o mejorar el modelo para balancear ambas métricas según el contexto clínico.

P50. Un banco quiere predecir impago de créditos. El dataset tiene 500k clientes, solo 12k no pagaron. ¿Qué problemas anticiparía y cómo los abordaría?

Problemas:

1. **Desbalance de clases** ($\approx 2,4\%$ positivos): el accuracy será engañoso. Usar F1, AUC-PR, ajustar `class_weight`.
2. **Valores faltantes** (8 % en ingreso, 3 % en historial): imputar correctamente, solo con datos de train.

3. **Variables categóricas** (empleo, zona, educación): aplicar encoding apropiado.
4. **Data leakage potencial**: variables que dependen del outcome (ej: monto total de deuda acumulada post-impago).

Métrica: AUC-PR o F1, no Accuracy.

Banco de preguntas completo — TD VI 1S-2026

Revise también los parciales modelo 1, 2 y 3 para ejercicios con gráficos y matrices de confusión.